Machine Learning Tutorial

Title: How Convolutional Neural Networks Extract Features from Images: A Layer-by-Layer Visual Analysis

Student Name: Menneni Ruchitha

StudentID: 24167049

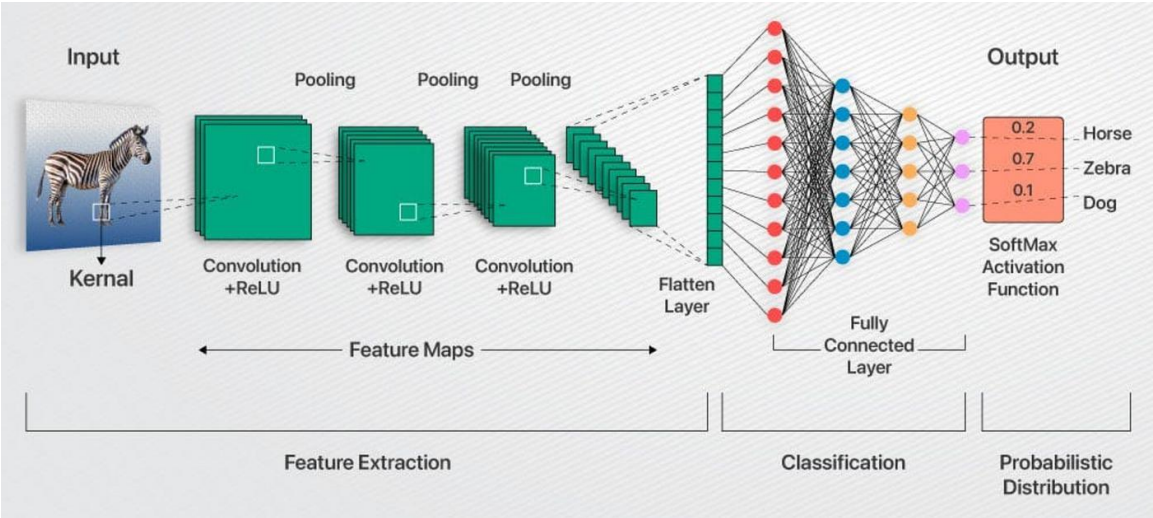
GitHub Repository: https://github.com/Ruchitha1107/cnn-feature-extraction/tree/main

Introduction

Convolutional Neural Networks (CNNs) are considered one of the most powerful tools used today in the field of machine learning, especially for image-related tasks such as classification, object detection, and segmentation. The main reason for the success of CNNs is the automatic extraction of relevant features from the image data, thus eliminating the need for any manual feature engineering.

In the conventional machine learning approaches, the features like edges, textures, or shapes were to be explicitly defined. However, in the CNN-based approaches, the features are learned directly from the data itself by a hierarchical process. This makes the CNN-based approaches more efficient for complex tasks, especially for images, for which the patterns are hard to define explicitly.

In this tutorial, we are going to focus on the understanding of the feature extraction process of images using CNNs. Instead of treating the CNNs as a black box, we are going to look at how each of the parts of the CNNs contributes to the process of feature extraction of the images. By the end of this tutorial, the reader should not only understand how the CNNs work, but also how to apply them.



2. The CNN Processing Pipeline

A CNN typically consists of a series of layers, where each layer is meant to recognize features of increasing complexity. The overall process can be described as:

Input Image -> Convolution -> Activation -> Pooling -> (Repeat) -> Fully Connected Layer -> Output

Each of these operations has a specific task

- Convolutional layers are used to recognize patterns like edges
- Activation functions are used to introduce non-linearity
- Pooling layers are used to reduce spatial dimensions

The overall concept here is that features become more abstract as we go deeper into the network.

3. Convolution Layer: Detecting Basic Features

The convolutional layer is the fundamental building block of a CNN, and it uses small matrices called filters or kernels to process the images.

The process of how a CNN works:

- The matrix, or filter, is moved over the image.
- Element-wise multiplication of the image and the matrix.
- The results are summed to produce a single number.
- This produces a new image, or feature map.

This data set division helps to ensure that the data set is tested on unseen data, thereby giving a true measure of the performance of the data set.

What Does It Learn?

Convolutional layers at early stages typically learn to recognize:

- Edges (horizontal, vertical, and/or diagonal)
- Gradients
- Textures

Some examples include:

- A vertical edge filter will respond to vertical edges
- A blur filter will smooth out images

Mathematical Intuition

If I is the input image and K is the kernel:

$$\text{Output}(x, y) = \sum_i \sum_j I(x+i, y+j) \cdot K(i, j)$$

This operation allows CNNs to detect spatial patterns.

4. Activation Function: Introducing Non-Linearity

After convolution, the output is passed through an activation function. The most commonly used activation in CNNs is the **ReLU (Rectified Linear Unit)**.

ReLU Function

$$f(x) = \max(0, x)$$

Why Is This Important?

Without activation functions:

- The network would behave like a linear model.
- The network would not be able to learn complex patterns.

ReLU helps by:

- Removing the negative values.
- Retaining the positive values.
- Making the model computationally efficient.

Effect on Feature Maps

Before ReLU:

- Feature maps contain both positive and negative values

After ReLU:

- Only strong (positive) activations remain
- Noise is reduced

5. Pooling Layer: Reducing Dimensionality

Pooling layers reduce the spatial size of feature maps, making computation more efficient and improving generalization.

Types of Pooling

- **Max Pooling** (most common): selects the maximum value
- **Average Pooling**: computes the average value.

The pooling layer is a key part of a Convolutional Neural Network (CNN) that helps reduce the size of feature maps while keeping the most important information. It works by sliding a small window, usually 2×2, across the feature map and summarising each region into a single value. One of the most commonly used methods is max pooling, where the highest value in each region is selected. This allows the network to keep the strongest features while ignoring less important details.

By reducing the size of the feature maps, pooling makes the model more efficient and lowers the computational cost. It also helps reduce overfitting by removing unnecessary or noisy information. Another important benefit is that pooling adds a level of translation invariance, meaning the model becomes less sensitive to small changes or shifts in the input image. Overall, pooling helps create feature maps that are smaller, more focused, and better suited for learning meaningful patterns.

Example (Max Pooling)

Input (2×2 region):

[1, 3]

[2, 4]

Output:

4

Why Pooling Matters

- Reduces computation
- Makes features more robust to small changes (translation invariance)
- Prevents overfitting

6. Layer-by-Layer Feature Extraction

This is the most important concept in understanding CNNs.

As data moves through the network, the type of features extracted evolves significantly.

Early Layers (Shallow Layers)

Early layers, also known as shallow layers, in a Convolutional Neural Network are responsible for detecting simple and fundamental features from the input image. These layers typically learn low-level patterns such as edges, lines, corners, and basic textures, which are common across most types of images. Since these features are not specific to any particular object, they form the building blocks for more complex feature extraction in later stages of the network. The filters in these layers respond strongly to basic visual structures, allowing the network to identify important boundaries and shapes within the image. This initial stage of feature extraction is crucial, as it provides the foundation upon which deeper layers can build more abstract and meaningful representations**.**

- Detect simple features
- Examples:
    - Edges
    - Lines
    - Corners

These features are similar across most images.

Middle Layers

Middle layers in a Convolutional Neural Network build upon the simple features detected in earlier layers to identify more complex patterns and structures. At this stage, the network begins to combine edges, lines, and textures to form meaningful shapes such as curves, contours, and repeated patterns. These layers capture more detailed information about the spatial arrangement of features, allowing the model to recognise parts of objects rather than just basic elements. Unlike shallow layers, the features learned here are more specific to the dataset and task. This intermediate level of representation plays a crucial role in bridging low-level feature detection and high-level understanding, enabling the network to gradually develop a deeper understanding of the input image.

- Combine simple features into more complex patterns
- Examples:
    - Textures
    - Repeated patterns
    - Basic shapes (circles, curves)

At this stage, the network starts recognizing **structure**.

Deep Layers (High-Level Features)

Deep layers, also known as high-level layers, in a Convolutional Neural Network are responsible for detecting complex and abstract features that are closely related to the final task, such as object recognition. At this stage, the network combines the patterns identified in earlier and middle layers to recognise meaningful structures like object parts or complete shapes, for example digits, faces, or specific objects. These features are highly specific to the dataset and the classification task being performed. Unlike earlier layers, which detect general patterns, deep layers focus on distinguishing between different classes by capturing unique characteristics. This allows the network to make accurate predictions, as it has learned a hierarchical representation that progresses from simple features to highly detailed and task-specific information.

- Detect complex, abstract features
- Examples:
    - Object parts (eyes, wheels, faces)
    - Class-specific patterns

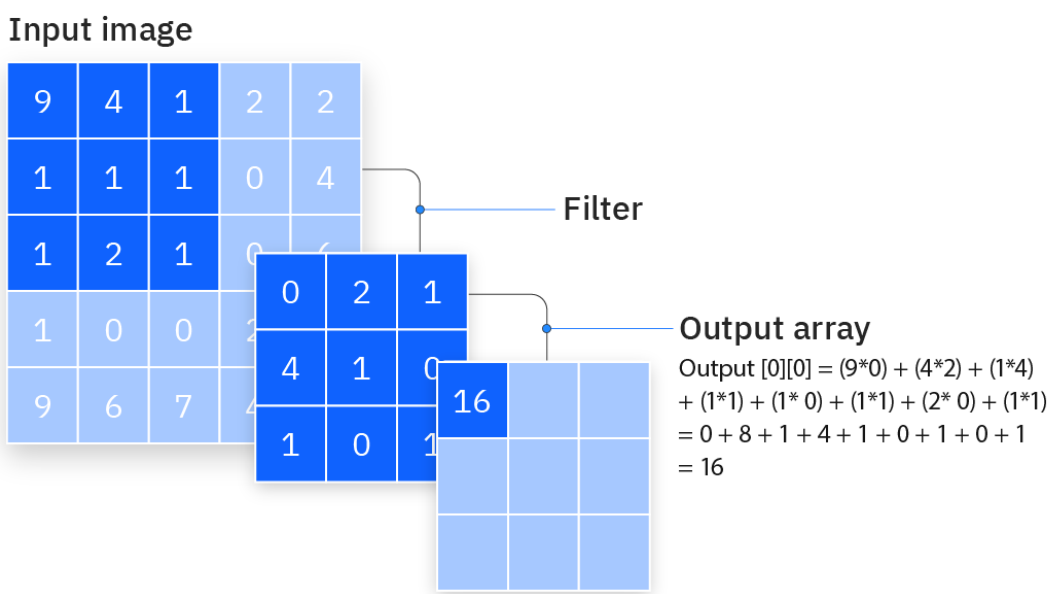
These features are highly task-specific.

Key Insight

CNNs build a **feature hierarchy**:

Pixels → Edges → Textures → Shapes → Objects

This hierarchical learning is what makes CNNs so powerful.



7. Strengths of CNNs

1. Automatic Feature Extraction

No need for manual feature engineering.

2. Spatial Awareness

CNNs preserve spatial relationships in images.

3. Parameter Sharing

Filters are reused, reducing the number of parameters.

4. High Performance

CNNs achieve state-of-the-art results in many tasks.

8. Limitations of CNNs

1. Data Requirements

CNNs require large datasets for effective training.

2. Computational Cost

Training deep CNNs can be expensive.

3. Lack of Interpretability

Despite visualization techniques, CNNs are still somewhat of a “black box.”

4. Sensitivity to Bias

If training data is biased, the model will learn those biases.

9. Conclusion

In conclusion, Convolutional Neural Networks are very powerful tools for image analysis, as demonstrated by the fact that they can automatically learn complex features from raw images. By looking at the entire network, we can understand how simple pixel information can be mapped to complex, high-level representations.

The entire process begins from the detection of simple edges, then proceeds to the detection of textures and shapes, and finally ends at the high-level detection of complex objects. This entire process can only be accomplished by the use of the layered approach, which enables the CNN to perform complex tasks with high accuracy.

It is very important to understand the internal mechanism of the CNN, as this will enable us to use the tools more effectively and understand the results obtained from the tools, as opposed to considering the tools as black boxes, as demonstrated by the visualization of feature extraction.

10. References (example – replace/add your own)

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition.
- Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Chollet, F. (2017). *Deep Learning with Python*.
- Stanford CS231n: Convolutional Neural Networks for Visual Recognition